

Лабораторная работа №13

Архитектура компьютеров

Безлепкина Т.И.

Российский университет дружбы народов, Москва, Россия

8 мая 2026

Информация

Докладчик

- ▶ Безлепкина Татьяна Игоревна
- ▶ Студентка НКАбд-01-25
- ▶ Таня
- ▶ Российский университет дружбы народов
- ▶ 1032253539@rudn.ru



Цель работы

Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задание

Задание

В ходе лабораторной работы требуется разработать четыре скрипта (и одну программу на C/C++):

- Командный файл с `getopts`: Скрипт анализирует ключи: `-inputfile` (входной файл), `-outputfile` (выходной файл), `-p` (шаблон поиска), `-C` (учёт регистра), `-n` (номера строк). На основе этих ключей с помощью `grep` выполняет поиск и выводит результат в файл или на экран.
- Программа на C/C++ и скрипт-анализатор: Программа вводит целое число и завершается с кодом `exit(1)`, `exit(2)` или `exit(0)` в зависимости от знака числа (плюс обработка ошибки ввода). Командный файл запускает эту программу, анализирует код возврата через `$?` и выводит сообщение: «число больше нуля», «меньше нуля» или «равно нулю».
- Создание и удаление нумерованных файлов: Скрипт принимает число `N` (аргумент командной строки) и создаёт `N` файлов с именами `1.tmp`, `2.tmp`, ..., `N.tmp`. При втором аргументе `delete` эти файлы удаляются (если существуют).
- Архивация изменённых файлов: Скрипт упаковывает в архив `tar` все файлы из заданной директории. В модифицированном варианте архивируются только файлы, изменённые менее недели назад (используется `find -mtime -7`).

Актуальность темы

Актуальность темы

UNIX-подобные системы широко используются на серверах и в облачных технологиях. Умение писать shell-скрипты с ветвлениями и циклами необходимо для автоматизации повседневных задач администратора и разработчика.

Объект и предмет исследования.

Объект и предмет исследования.

Объект — командный процессор ОС UNIX (bash). Предмет — управляющие конструкции: ветвления (if, case), циклы (for, while, until), обработка аргументов (getopts) и коды возврата программ.

Научная новизна.

Научная новизна.

Работа имеет учебный характер. Новизна заключается в систематизации приёмов использования ветвлений и циклов при решении типовых задач обработки файлов и текста в ОС UNIX.

Практическая значимость работы.

Практическая значимость работы.

Разработанные скрипты (поиск строк, создание файлов, архивация свежих данных) могут применяться на практике для автоматизации рутинных операций. Полученные навыки являются основой для дальнейшего изучения администрирования UNIX-систем.

Теоретическое введение

Теоретическое введение

В операционной системе UNIX командный процессор (оболочка, shell) является не только средством для выполнения команд, но и мощным языком программирования. Shell-скрипты позволяют автоматизировать повторяющиеся задачи, управлять файлами и процессами, использовать условные операторы, циклы, функции и обрабатывать аргументы командной строки. В данной лабораторной работе особое внимание уделяется: Разбору ключей командной строки с помощью встроенной команды `getopts`, которая позволяет гибко обрабатывать опции и их аргументы. - Ветвлениям – конструкциям `if-then-else`, `case` и проверке кодов возврата программ (переменная `$?`). - Циклам – `for`, `while`, `until`, их синтаксису и отличиям. - Генерации имён файлов с помощью метасимволов (`*`, `?`, `[]`), которые подставляются оболочкой. - Взаимодействию с программами на языках C/C++ через коды завершения `exit()`. - Архивации файлов командами `tar` и `find`, в том числе с фильтрацией по времени изменения. - Эти знания необходимы для написания эффективных и надёжных командных файлов в среде UNIX.

Выполнение лабораторной работы

Выполнение лабораторной работы

На рисунке 1 представлен листинг командного файла search.sh. В нём используются конструкции getopt для разбора ключей -i, -o, -p, -C, -n, а также условный оператор if для проверки существования входного файла и перенаправление вывода grep. Скрипт реализует поиск строк по шаблону с возможностью учёта регистра и вывода номеров строк.

```
GNU nano 8.5 search.sh
# Переменные
inputfile=""
outputfile=""
pattern=""
case_sensitive="" # будет "-i" при ключе -C, иначе пусто
line_numbers=""

# Разбор ключей
while getopt "i:o:p:Cn" opt; do
    case $opt in
        i) inputfile="$OPTARG" ;;
        o) outputfile="$OPTARG" ;;
        p) pattern="$OPTARG" ;;
        C) case_sensitive="-i" ;; # -C включает игнорирование регистра
        n) line_numbers="-n" ;;
        *) echo "Неверный ключ. Используйте: $0 -i файл -p шаблон [-o выходной] [-C] [-n]"
            exit 1 ;;
    esac
done

# Проверка обязательных аргументов
if [ -z "$inputfile" ] || [ -z "$pattern" ]; then
    echo "Ошибка: нужно указать -i и -p"
    exit 1
fi

# Проверка существования входного файла
if [ ! -f "$inputfile" ]; then
```

На рисунке 2 показан процесс выполнения скрипта. Командой `chmod +x search.sh` заданы права на выполнение. Затем скрипт запущен с ключами: `-i test.txt -p "Hi" -C -n -o result.txt`. В результате в файл `result.txt` выведены строки, содержащие "Hi" без учёта регистра, с номерами строк.

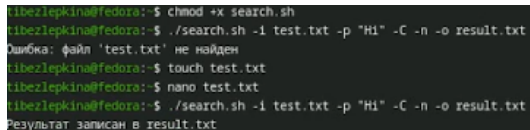
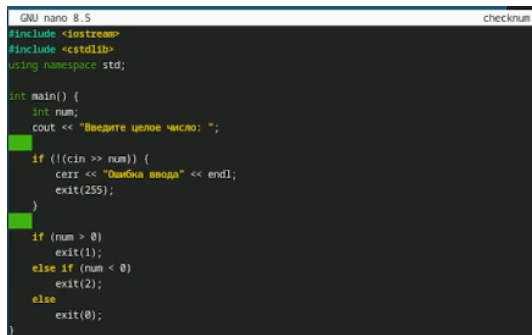
A screenshot of a terminal window with a dark background and light green text. The prompt is 'tibezelepkina@fedora:~\$'. The user enters 'chmod +x search.sh'. The prompt changes to 'tibezelepkina@fedora:~\$' and the user enters './search.sh -i test.txt -p "Hi" -C -n -o result.txt'. The output is 'Ошибка: файл 'test.txt' не найден'. The user enters 'touch test.txt'. The prompt changes to 'tibezelepkina@fedora:~\$' and the user enters 'nano test.txt'. The prompt changes to 'tibezelepkina@fedora:~\$' and the user enters './search.sh -i test.txt -p "Hi" -C -n -o result.txt'. The output is 'Результат записан в result.txt'.

Figure 2: процесс выполнения скрипта для 1 задания

Рисунок 3 — листинг checknum.cpp. Программа вводит число и завершается с `exit(1)` если больше 0, `exit(2)` если меньше 0, `exit(0)` если равно 0, `exit(255)` при ошибке ввода.



```
GNU nano 8.5 checknum.cpp
#include <iostream>
#include <cstdlib>
using namespace std;

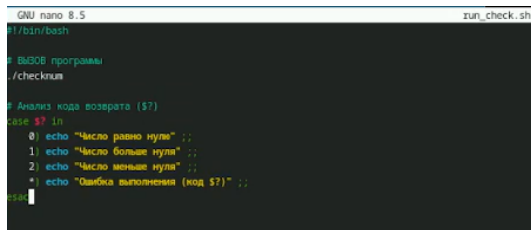
int main() {
    int num;
    cout << "Введите целое число: ";

    if (!(cin >> num)) {
        cerr << "Ошибка ввода" << endl;
        exit(255);
    }

    if (num > 0)
        exit(1);
    else if (num < 0)
        exit(2);
    else
        exit(0);
}
```

Figure 3: листинг checknum.cpp

Рисунок 4 — листинг run_check.sh. Скрипт запускает checknum, анализирует \$? через case и выводит сообщение.



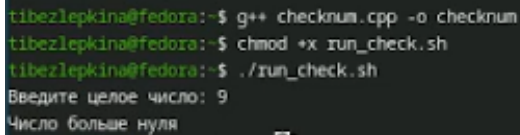
```
GNU nano 8.5 run_check.sh
#!/bin/bash

# Вывод программы
./checknum

# Анализ кода возврата ($?)
case $? in
  0) echo "Число равно нулю" ;;
  1) echo "Число больше нуля" ;;
  2) echo "Число меньше нуля" ;;
  *) echo "Ошибка выполнения (код $?)" ;;
esac
```

Figure 4: листинг run_check.sh

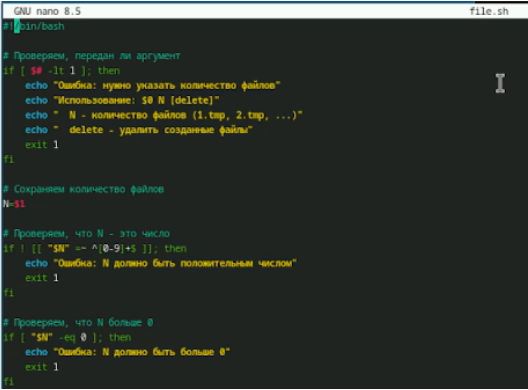
Рисунок 5 — выполнение. Компиляция g++ checknum.cpp -o checknum, запуск run_check.sh. При вводе 5 — «больше нуля», -2 — «меньше нуля», 0 — «равно нулю», abc — «ошибка».

A terminal window with a dark background and green text. It shows the execution of a C++ program and a shell script. The user enters '5' and the program outputs 'Число больше нуля'.

```
tibezlepkina@fedora:~$ g++ checknum.cpp -o checknum
tibezlepkina@fedora:~$ chmod +x run_check.sh
tibezlepkina@fedora:~$ ./run_check.sh
Введите целое число: 9
Число больше нуля
```

Figure 5: процесс выполнения скрипта для 2 задания

Рисунок 6 — листинг files.sh. Скрипт создаёт файлы 1.tmp ... N.tmp через цикл for, если второй аргумент delete — удаляет.



```
GNU nano 8.5 file.sh
#!/bin/bash

# Проверяем, передан ли аргумент
if [ $# -lt 1 ]; then
    echo "Ошибка: нужно указать количество файлов"
    echo "Использование: $0 N [delete]"
    echo "  N - количество файлов (1.tmp, 2.tmp, ...)"
    echo "  delete - удалить созданные файлы"
    exit 1
fi

# Сохраняем количество файлов
N=$1

# Проверяем, что N - это число
if ! [[ "$N" =~ ^[0-9]+$ ]]; then
    echo "Ошибка: N должно быть положительным числом"
    exit 1
fi

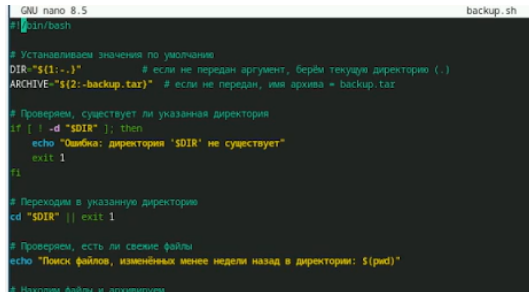
# Проверяем, что N больше 0
if [ "$N" -eq 0 ]; then
    echo "Ошибка: N должно быть больше 0"
    exit 1
fi
```

Figure 6: листинг files.sh

Рисунок 7 — выполнение. ./files.sh 5 — созданы пять файлов, ls проверяет. ./files.sh 5 delete — файлы удалены.

```
tibezlepkina@fedora:~$ nano file.sh
tibezlepkina@fedora:~$ chmod +x file.sh
tibezlepkina@fedora:~$ ./file.sh 5
Создаю 5 файлов...
Создан 1.tmp
Создан 2.tmp
Создан 3.tmp
Создан 4.tmp
Создан 5.tmp
Готово. Создано 5 файлов.
tibezlepkina@fedora:~$ ./file.sh 5 delete
Удаляю файлы...
Удалён 1.tmp
Удалён 2.tmp
Удалён 3.tmp
Удалён 4.tmp
Удалён 5.tmp
Готово. Удаление завершено.
tibezlepkina@fedora:~$
```

Рисунок 8 — листинг backup.sh. Скрипт переходит в каталог, `find -mtime -7` ищет файлы новее 7 дней, `tar` создаёт архив.

A screenshot of a terminal window with a dark background. The title bar at the top shows 'GNU nano 8.5' on the left and 'backup.sh' on the right. The script content is as follows:

```
#!/bin/bash

# Устанавливаем значения по умолчанию
DIR="${1:-.}" # если не передан аргумент, берём текущую директорию (..)
ARCHIVE="${2:-backup.tar}" # если не передан, имя архива = backup.tar

# Проверяем, существует ли указанная директория
if [ ! -d "$DIR" ]; then
    echo "Ошибка: директория '$DIR' не существует"
    exit 1
fi

# Переходим в указанную директорию
cd "$DIR" || exit 1

# Проверяем, есть ли свежие файлы
echo "Поиск файлов, изменённых менее недели назад в директории: $(pwd)"

# Находим файлы и архивируем
```

Figure 8: листинг backup.sh

Рисунок 9 — процесс выполнения скрипта для 4 задания. Скрипту даны права на выполнение, запуск с указанием директории и имени архива. В архив попали только файлы, изменённые менее 7 дней назад.

```
tibezlepkina@fedora:~$ chmod +x backup.sh
tibezlepkina@fedora:~$ ./backup.sh
Поиск файлов, изменённых менее недели назад в директории: /home/tibezlepkina
Архив успешно создан: /home/tibezlepkina/backup.tar
Содержимое архива:
./mozilla/extensions/(ec8030f7-c20a-464f-9b0e-13a3a9e97384)/.fedora-langpack-install
./mozilla/firefox/Crash Reports/crash_helper_server.log
./mozilla/firefox/Crash Reports/LastCrash
./mozilla/firefox/Crash Reports/submit.log
./mozilla/firefox/Crash Reports/pending/7e308349-0fc1-eee4-4383-2d0cbaf97b7d.dmp
./mozilla/firefox/Crash Reports/pending/7e308349-0fc1-eee4-4383-2d0cbaf97b7d.extra
./mozilla/firefox/Crash Reports/pending/3a3df0b9-deld-4659-a5e4-3d40ec71d67a.dmp
./mozilla/firefox/Crash Reports/pending/3a3df0b9-deld-4659-a5e4-3d40ec71d67a.extra
./mozilla/firefox/Crash Reports/pending/5aea50ec-31d3-3b4a-cfdb-b2140e5bbb6f.dmp
./mozilla/firefox/Crash Reports/pending/5aea50ec-31d3-3b4a-cfdb-b2140e5bbb6f.extra
./mozilla/firefox/Crash Reports/pending/55d7edad-348d-c84f-8127-b3310505fd30.dmp
./mozilla/firefox/Crash Reports/pending/55d7edad-348d-c84f-8127-b3310505fd30.extra
./mozilla/firefox/Crash Reports/pending/55462aea-2a44-93ea-949b-a4554ca59d18.dmp
./mozilla/firefox/Crash Reports/pending/55462aea-2a44-93ea-949b-a4554ca59d18.extra
./mozilla/firefox/Crash Reports/pending/3f57c7c0-c5ee-6ef9-90f7-7b1aa9733173.dmp
```

Figure 9: процесс выполнения скрипта для 4 задания

Выводы

Выводы

В ходе выполнения лабораторной работы были изучены и практически применены основные конструкции программирования в командном процессоре UNIX: разбор опций с помощью `getopts`, условные операторы `if` и `case`, циклы `for` и `while`, работа с кодами возврата (`$?`), генерация имён файлов с метасимволами, а также утилиты `grep`, `tar`, `find`. Полученные навыки позволяют автоматизировать типовые задачи администрирования и обработки данных в среде UNIX. Использование команды `exit(n)` в программе на C/C++ продемонстрировало механизм взаимодействия между скомпилированной программой и оболочкой через код завершения.

Список литературы